

T3.1 Arrays

Ejercicio 1. Diagonal de un array 3x3

Crea un array de 3x3, pide al usuario, en nueve entradas diferentes, que introduzca un número entero para ir rellenando el array por orden (filas y columnas). Muestra por consola, en tres salidas, aquellos que forman parte de la diagonal.

Las nueve entradas rellenan el array fila a fila: las tres primeras son la primera fila, las tres siguientes la segunda y las tres últimas la tercera. Muestra un número por línea, cada uno con su propio console.log (no uses saltos de línea \n), en el orden de la diagonal principal (primera fila, segunda fila, tercera fila).

Ejercicio 2. Índices de los true

Crea un programa que almacene un array unidimensional de 10 elementos, se pedirá en 10 entradas diferentes al usuario que introduzca "true" o "false" para almacenar en el array su Boolean correspondiente por orden. Luego muestra por consola, en salidas diferentes, el/los indexes que ocupa cada elemento true.

Muestra un índice por línea, cada uno con su propio console.log (no uses saltos de línea \n), de menor a mayor, empezando en 0. Si ningún elemento es true no se muestra nada.

Ejercicio 3. Posiciones de la letra a con indexOf

Con la base de un array ["a", "a", "i", "a", "e", "i", "a"] Utiliza indexOf para obtener todas las posiciones en las que se encuentra la letra "a", contando las posiciones desde 1 y no desde 0: el primer elemento del array es la posición 1, el segundo la 2, y así sucesivamente. Imprime por consola el array resultante.

El programa no pide datos. Muestra el array de posiciones en una sola línea con console.log; para el array de la base el resultado es [1,2,4,7].

Ejercicio 4. Número de repeticiones

Crea un programa que pida al usuario introducir 10 elementos en un array unidimensional y devuelva por consola el Number de cuantas repeticiones hay en el array. Por ejemplo, [1,1,2,2,3,4,5,6,7,8] devolvería 4.

Los 10 elementos se piden en 10 entradas diferentes. Se cuenta cada elemento cuyo valor aparece más de una vez en el array: en el ejemplo, los dos 1 y los dos 2 suman 4. Muestra sólo el número.

Ejercicio 5. Elemento más repetido

Crea un programa que pida al usuario introducir 10 elementos en un array unidimensional y devuelva por consola el elemento que más veces aparece repetido, en caso de que haya dos o más elementos que son los más repetidos y tienen el mismo número de repeticiones devolverá "error".

Los 10 elementos se piden en 10 entradas diferentes. Si los diez elementos son distintos, todos empatan con una aparición y también se muestra "error". Muestra sólo el elemento (o "error") en una línea.

Ejercicio 6. Sin repetidos, conservando el último

Crea un programa que pida al usuario introducir en 10 entradas diferentes, 10 elementos en un array unidimensional y devuelva por consola el array resultante sin números repetidos, manteniendo siempre el último valor y eliminando los anteriores. No utilices conjuntos ni el método filter. Por ejemplo: [1,2,3,4,5,5,4,3,2,1] como entrada devolvería [5,4,3,2,1]

Muestra el array resultante en una sola línea con console.log.

Ejercicio 7. Jugadores online

Crea un programa de estado de un juego online, el programa debe pedir el usuario que introduzca por consola, separado por espacios, el nombre / los nombres de usuario de uno o varios jugadores, comenzando por @. El programa tiene que devolver el estado de cuantos jugadores están online con el siguiente formato:

1. "@chicharito99 está online" en caso de que sólo haya un jugador conectado
2. "@fuegobasket08 y @chicharito99 están online" en caso de que haya 2 jugadores conectados, se mostrarán por orden decreciente que se conectaron (primero se conectó @chicharito99 y después @fuegobasket08)
3. "@pepe, @fuegobasket y 1 persona(s) más están online" en caso de que haya más personas conectadas, otra vez por orden decreciente que se conectaron, siendo el último en conectarse en este caso @pepe.

En caso de que el nombre de usuario no comience por @ no se tendrá en cuenta.

Los nombres llegan en una única entrada, en el orden en que se conectaron (el primero de la entrada es el primero que se conectó). El mensaje se muestra en una sola línea.

Ejercicio 8. Patrón con un array

Crea un programa que, con un único bucle for, y dos if (se considera el método switch y la condición ternaria como un símil), muestre por consola el siguiente patrón:

```
[1]
[1,2]
[1,2,3]
[1,2,3,4]
[1,2,3,4,5]
[1,2,3,4,5,6]
[1,2,3,4,5,6,7]
[1,2,3,4,5,6,7,8]
[1,2,3,4,5,6,7,8,9]
[1,2,3,4,5,6,7,8,9,0]
[1,2,3,4,5,6,7,8,9,0,9]
[1,2,3,4,5,6,7,8,9,0,9,8]
[1,2,3,4,5,6,7,8,9,0,9,8,7]
[1,2,3,4,5,6,7,8,9,0,9,8,7,6]
[1,2,3,4,5,6,7,8,9,0,9,8,7,6,5]
[1,2,3,4,5,6,7,8,9,0,9,8,7,6,5,4]
[1,2,3,4,5,6,7,8,9,0,9,8,7,6,5,4,3]
[1,2,3,4,5,6,7,8,9,0,9,8,7,6,5,4,3,2]
[1,2,3,4,5,6,7,8,9,0,9,8,7,6,5,4,3,2,1]
[2,3,4,5,6,7,8,9,0,9,8,7,6,5,4,3,2,1]
[3,4,5,6,7,8,9,0,9,8,7,6,5,4,3,2,1]
[4,5,6,7,8,9,0,9,8,7,6,5,4,3,2,1]
[5,6,7,8,9,0,9,8,7,6,5,4,3,2,1]
[6,7,8,9,0,9,8,7,6,5,4,3,2,1]
[7,8,9,0,9,8,7,6,5,4,3,2,1]
[8,9,0,9,8,7,6,5,4,3,2,1]
[9,0,9,8,7,6,5,4,3,2,1]
[0,9,8,7,6,5,4,3,2,1]
[9,8,7,6,5,4,3,2,1]
[8,7,6,5,4,3,2,1]
[7,6,5,4,3,2,1]
[6,5,4,3,2,1]
[5,4,3,2,1]
[4,3,2,1]
[3,2,1]
[2,1]
[1]
```

Siendo lo que se muestra en todo momento el contenido de un array añadiendo elementos y eliminándolos. Para imprimir en la misma línea los String utiliza el método `stringify()` de la librería JSON, así no se modifica el contenido del array. No se permite imprimir el patrón directamente (escribir las líneas a mano o como texto): cada línea tiene que ser el array impreso en ese momento.

El programa no pide datos. Son 37 líneas, una por cada estado del array, cada una con su propio `console.log` (no uses saltos de línea `\n`).

Ejercicio 9. Alta de usuarios por DNI

Sin utilizar el método `slice()`. Crea un programa que añada nuevos usuarios utilizando Arrays. Por consola pedirá, en la misma entrada separada por espacios, el nombre, primer apellido, segundo apellido y número del DNI (con su letra en mayúscula), siendo el número del DNI la clave. Considera que siempre se introduce un nombre con dos apellidos en formato String, sin problemas de formatos.

En caso de que la clave ya exista devolver "DNI ya existe" y no introducir el nuevo elemento del array; en caso de que se introduzca un DNI con la letra incorrecta devolver "Error DNI" y no introducir el nuevo elemento en el array; en caso de que se introduzca correctamente el nombre, dos apellidos y dni devolver el array completo.

Tras cualquiera de estas acciones volver a pedir la introducción de datos, y únicamente salir del programa cuando se introduzca como entrada "exit".

La letra del DNI es la que ocupa, en la cadena "TRWAGMYFPDXBNJZSQVHLCKE", la posición del resto de dividir el número entre 23. La letra tiene que ir en mayúscula: una letra en minúscula se considera incorrecta y se muestra "Error DNI". Cada entrada produce exactamente una línea de salida: el mensaje de error o el array completo de usuarios mostrado con `console.log` (la estructura interna del array la decides tú, pero en la línea tienen que aparecer el nombre, los apellidos y el DNI de cada usuario dado de alta). La entrada "exit" no muestra nada.

Ejercicio 10. Mini-buscaminas

Crea un programa con un array bidimensional que represente el tablero de un juego de mini-buscaminas de 10 x 10 que siga los siguientes pasos:

1. Inicialización del tablero:

- Se debe inicializar el array del tamaño definido.
- Solicitar al usuario que introduzca, en la misma entrada, separada por una coma, la coordenada en la que habrá una mina. Ejemplo: "3,3"
- Si se introduce una coordenada fuera del array el juego finalizará con el mensaje "Error de juego"

2. Juego:

- El jugador ingresa en una entrada la coordenada con el mismo formato que anteriormente.
- Si la coordenada ingresada contiene una mina, el programa finaliza y se debe mostrar un mensaje antes indicando "Game over"
- Si la coordenada ingresada contiene una casilla vacía, se debe mostrar un mensaje indicando que puede continuar jugando, y el programa debe permitir ingresar nuevas coordenadas: en caso de que no haya ninguna mina en sus casillas adyacentes el programa devolverá en una línea "Frío, frío" y a continuación volverá a solicitar otra entrada. En caso de que haya una mina en una de sus casillas adyacentes mostrará "Caliente, caliente" y a continuación volverá a solicitar otra entrada.
- Las casillas ya descubiertas no deben poderse seleccionar de nuevo, por lo que no se devolverá ningún mensaje nuevo, sólo se volverá a pedir una coordenada.
- En caso de que el jugador descubra todas las casillas vacías se debe mostrar un mensaje que indique "Has ganado, enhorabuena!" y finalice el programa.

Las coordenadas van de 0 a 9 en cada eje. El mensaje que indica que se puede seguir jugando es "Frío, frío" o "Caliente, caliente", no hace falta ningún otro. Las casillas adyacentes son las ocho que rodean a una casilla (también las diagonales). Los mensajes "Frío, frío", "Caliente, caliente", "Game over", "Has ganado, enhorabuena!" y "Error de juego" se muestran cada uno en su propia línea, con su propio console.log (no uses saltos de línea \n) y con ese texto exacto.

Ejercicio 11. Gestor de tareas

Crea un programa de gestión de tareas donde cada tarea estará representada por un array con el siguiente formato: ["título", "descripción", realizada], siendo título y descripción Strings, y realizada un Boolean (true si la tarea está completada y false si no lo está).

El programa debe mostrar al usuario un menú con las siguientes opciones (solo se mostrarán las 5 opciones, una por línea), y a continuación un prompt con el string "Elige una opción (1-5): " donde puede interactuar mediante entradas numéricas:

1. Agregar tarea:

- El programa debe pedir al usuario que ingrese el título y la descripción de la nueva tarea en dos entradas diferentes de consola.
- La tarea agregada por defecto debe tener el estado de no realizada (realizada = false).
- Validar que el título y la descripción no estén vacíos. Si alguna de estas entradas es inválida, se almacenará la tarea con título "error", descripción "error" y realizada false.
- No mostrar ningún tipo de mensaje de confirmación, únicamente añadir.

2. Eliminar tarea:

- El programa mostrará una lista numerada de las tareas actuales, mostrando únicamente el número (empezando en 1), el título y el estado, separadas por un espacio. Ej. `console.log(`${index + 1} ${nombreTarea} ${estadoTarea}`);`
- El usuario deberá ingresar el número de la tarea que desea eliminar, siendo la primera tarea la número 1.
- Tras eliminar una tarea, mostrar "Tarea \${nombreTarea} eliminada" confirmando la eliminación.

3. Marcar tarea como completada:

- El programa mostrará una lista numerada de las tareas actuales, con el mismo formato que en la opción 2.
- El usuario debe seleccionar el número de la tarea que desea marcar como completada, siendo la primera tarea la número 1.
- Si la tarea seleccionada ya está marcada como completada, se debe mostrar un mensaje indicando "Tarea \${nombreTarea} ya completada" y devolver al usuario al menú principal.
- Validar que el número ingresado sea correcto (debe corresponder a una tarea válida), en caso contrario devolver "Tarea inexistente" y volver al menú principal.

4. Mostrar solo las tareas completadas:

- Se mostrará una lista de todas las tareas cuyo estado sea true.
- Si no hay tareas completadas, se mostrará un mensaje indicando "No existen tareas completadas".
- El formato de la salida debe ser un `console.log` de cada subarray que cumpla las condiciones (`console.log(tarea);`).

5. Salir del programa:

- Al elegir esta opción, el programa debe mostrar por consola "Hasta pronto" y finalizar la ejecución.

Validaciones:

- Menú principal: cualquier opción que no sea un número entre 1 y 5 debe generar un mensaje de "error". El menú se volverá a mostrar hasta que el usuario ingrese una opción válida. Aceptar solo entradas numéricas.

- Agregar tarea: el programa debe asegurarse de que no haya títulos duplicados. Si un título ya existe, se debe mostrar "Error. Título duplicado" y volver a mostrar el menú principal, sin añadir ninguna tarea nueva.
- Eliminar tarea: si no hay tareas en la lista, debe mostrar "No hay tareas para eliminar". El número ingresado debe estar dentro del rango de las tareas disponibles; si está fuera de rango, mostrar "Error de rango" y volver al menú principal.
- Marcar tarea como completada: si no hay tareas en la lista, debe mostrar "No hay tareas para marcar como completadas".

Cada línea de la salida (cada opción del menú, cada tarea de las listas y cada mensaje) se muestra con su propio `console.log`: no uses saltos de línea `\n`. El menú son exactamente cinco líneas de `console.log` (una por opción) y se vuelve a mostrar completo cada vez que se regresa a él; el texto de cada opción lo decides tú. El estado en las listas numeradas se muestra como `true` o `false`.